Google Summer of Code 2026

CERN-HSF

AI-Assisted PostgreSQL Optimization Using EXPLAIN Plans

Mentor: Ruslan Mashinistov

Project Size : Large (350 hours)

Sarthak Negi

Github • CV • Portfolio

Table of Content

1. Introduction **1**
Prior Engagement 2

2. About the Project **3**
Three Query Strategies for the Same Endpoint 3
Current Architecture 4
The Infrastructure Gap 4

3. Problem to be Addressed **6**

4. Proposed Solution **7**

5. Projects Goals **9**

6. Implementation Plan **9**
First Half — Infrastructure, Observability, and EXPLAIN Collection 9
Phase 1: PostgreSQL Cluster and Replica Infrastructure 9
Phase 2: Monitoring and Observability Stack 10
Phase 3: EXPLAIN Plan Collection Pipeline 10

Second Half — AI-Assisted Analysis Engine 11
Phase 5: LLM Integration, Dynamic Tuner, and Suggestion API 12
Phase 6: Staging Validation and Documentation 13

6.1 Project Methodology 14

7. Technical Elements **15**
7.1 PostgreSQL StatefulSet and Replica 15
7.2 Monitoring Stack 17
7.3 EXPLAIN Plan Collection Pipeline 18
7.4 AI Analysis Engine 20
7.5 Dynamic Parameter Tuner 24
7.6 Suggestion REST API 24

8. Challenges and Proposed Solutions **25**

9. Timeline **28**
Availability 30
Post-GSoC Plans and Community Engagement 31

10. Biographical Information **31**
Educational Background 31
Relevant Experience 31
Personal Information 34

11. References **35**

1. Introduction

High-energy physics experiments generate and rely on massive volumes of calibration data over their lifetimes. The systems managing this data must be not only correct, but also highly performant—a slow calibration lookup can stall an entire reconstruction pipeline running across thousands of compute nodes

But raw signals alone are not enough. Every measurement depends on the conditions under which it was taken: collider configuration, detector geometry and alignment, magnetic field maps, electronics settings, and environmental parameters like temperature and humidity. These are called Conditions Data, and they directly affect how experimental data is processed and interpreted.

Because conditions change over time — through aging, maintenance, or reconfiguration — each set of conditions must be tagged with an Interval of Validity (IOV). Consistent groups of IOV-tagged conditions are bundled into Global Tags, representing complete snapshots of the experimental setup for real or simulated running periods.

A Conditions Database (CDB) manages this relationship at scale. The nopayloaddb project serves as the HEP Software Foundation's reference [1] implementation of a Conditions Database, and is currently used to power calibration lookups for experiments such as sPHENIX at Brookhaven National Laboratory and Belle II at KEK.

Prior Engagement

Before writing this proposal, I ran nopayloaddb locally using docker-compose, populated the database with the provided test.py script, and exercised the live API:

```
curl
"http://localhost:8000/api/cdb_rest/payloadiovs/?gtName=generic_gt&majorIOV=10&minorIOV=10"
```

The returned payload confirmed the data structure end-to-end:

```
[["GenericModule", "/example/payload1.root", "dummy_checksum", null, 0, 0, 1, 11]]
```

I traced the Django log to capture the full SQL sent to PostgreSQL, identified the four-table LATERAL JOIN (GlobalTag → PayloadList → PayloadIOV → PayloadType) with the comb_iov decimal range filter, and analyzed the query plan. From that hands-on work I formed the following performance hypotheses — which directly inform the optimization rules the AI engine in this proposal will implement:

1. Covering index extension for index-only scans — The current covering_idx covers (payload_list_id, comb_iov DESC NULLS LAST), but the LATERAL subquery also fetches payload_url, checksum, size, major_iov, minor_iov, major_iov_end, minor_iov_end. Adding those columns to the index would eliminate heap fetches entirely on the hot path.

2. Materialized view per locked GlobalTag — Once a GlobalTag reaches status='locked' its IOVs never change. Pre-computing the "latest valid IOV per PayloadType" as a materialized view turns the join chain into a simple indexed lookup.

3. Redis application-layer cache — The cache key (gtName, majorIOV, minorIOV) maps to a stable result for locked tags. With a long TTL this eliminates database hits for repeated identical lookups (e.g., the same run queried by thousands of reconstruction jobs).

4. PayloadIOV partitioning — At hundreds of millions of rows, hash-partitioning by payload_list_id would align with the covering index key and distribute load evenly. This is only worth the operational complexity at that scale.

These four ideas are not speculative—they come directly from analyzing real EXPLAIN plan output for this query. Each is formalized as a named detection rule in the **AI engine described in Section 7.**

Despite this groundwork, the database layer currently runs as a single PostgreSQL instance. There is no systematic query observability, no cluster-level redundancy, and no automated feedback loop to detect and correct performance regressions. In contrast, [1] the application layer is already designed for a multi-replica setup: the sPHENIX and Belle2 HSF Helm charts define `POSTGRES_HOST_W`, `POSTGRES_HOST_R1`, and `POSTGRES_HOST_R2`, and route all traffic through PgBouncer—yet no actual PostgreSQL replicas exist to support this architecture.

This proposal focuses on closing that gap. Over 24 weeks, I will build an AI-assisted PostgreSQL optimization framework that operates in a fully non-invasive manner—adding new infrastructure and tooling around the existing system without requiring changes to tables, migrations, or Helm templates.

The framework will provision a production-grade PostgreSQL cluster with streaming replication [5] on OpenShift using Helm, integrate comprehensive query monitoring into the existing Promtail/Loki pipeline, automatically collect and analyze EXPLAIN plans [3] from slow queries, and leverage a layered AI engine to generate actionable, operator-reviewed optimization recommendations for dynamic cluster tuning.

2. About the Project

What Nopayloaddb Does

Nopayloaddb [2, 7] is a RESTful conditions database built on Nginx + Django 4.2 + PostgreSQL 16, deployed on BNL's OKD cluster at SDCC. Experiments such as sPHENIX use it to retrieve time-varying calibration objects — magnetic field maps, detector thresholds, geometry parameters — keyed by a Global Tag and an Interval of Validity (IOV).

A representative production API call:

```
GET /api/cdb_rest/payloadiovs/?gtName=sPHENIX_ExampleGT_24&majorIOV=0&minorIOV=999999
```

This returns, for every payload type attached to that Global Tag, the single most-recent calibration entry valid at the requested IOV. The official Nopayloaddb documentation [7] explicitly lists [2] read replicas and Redis-based caching as planned performance enhancements for this endpoint — both of which this project will implement as AI-detected recommendations.

Three Query Strategies for the Same Endpoint

The codebase provides three alternative implementations of the IOV lookup, each with different performance trade-offs. All three are live in production and all three will be benchmarked and monitored:

View Class	Endpoint	Strategy
PayloadIOVsSQLListAPIView	GET /api/cdb_rest/payloadiovs/	Raw SQL with LATERAL JOIN + comb_iov decimal range
PayloadIOVsORMOrderByListAPIView	GET /api/cdb_rest/payloadiovs_orm_orderby/	ORM with .order_by('payload_list_id', '-comb_iov').distinct('payload_list_id')
PayloadIOVsORMMaxListAPIView	GET /api/cdb_rest/payloadiovs_orm_max/	ORM with Max('comb_iov') + dynamic Q statement

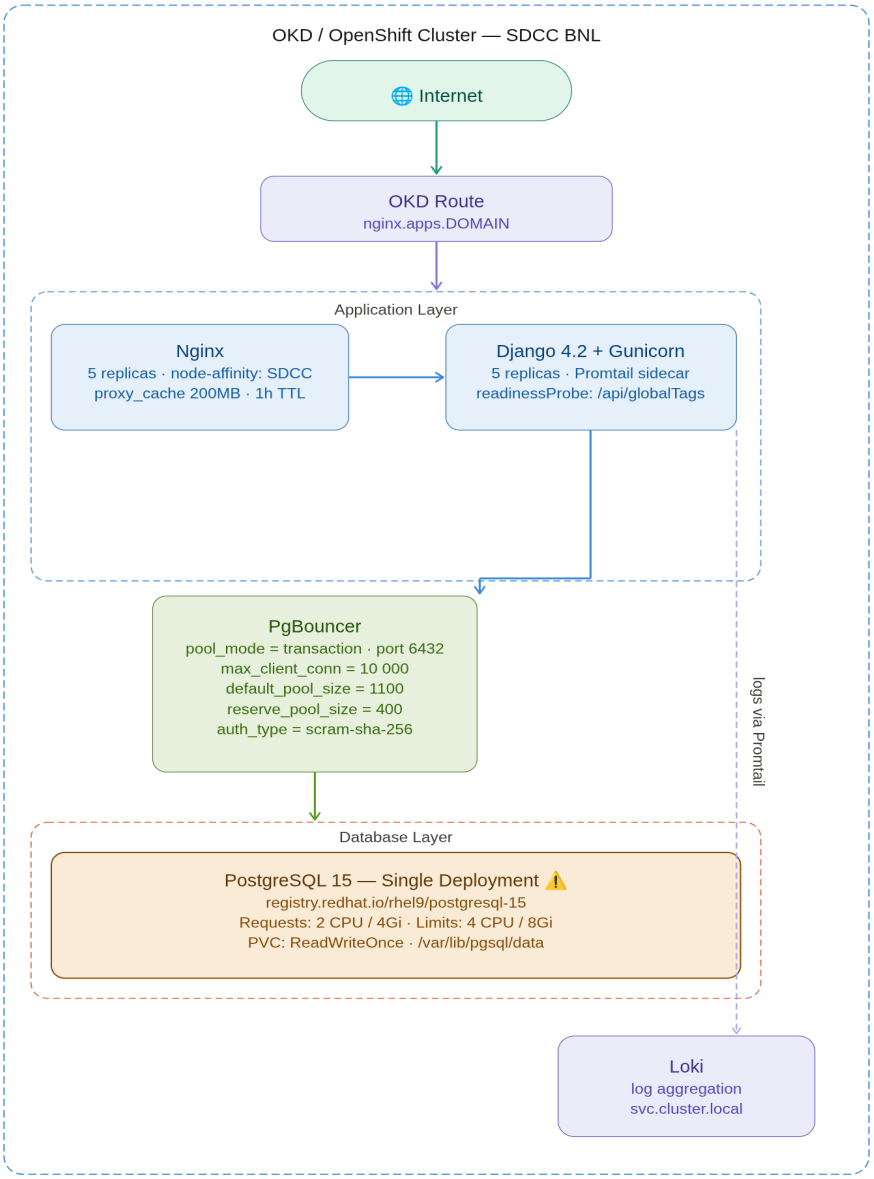
The AI engine will collect EXPLAIN plans [3] from all three, identify which strategy is in use for a given workload, and — if an experiment is calling the slower ORM endpoints at high volume — surface a recommendation to route to the SQL endpoint instead.

Additionally:

- GET /api/cdb_rest/payloadiovs_ranges/ — range-based IOV queries via PayloadIOVsRangesListAPIView
- POST /api/cdb_rest/bulk_piov — bulk PayloadIOV creation via PayloadIOVBulkCreationAPIView
- POST /api/cdb_rest/cloneGlobalTag/{source}/{target} — deep copy of a GT via GlobalTagCloneAPIView (@transaction.atomic)
- PUT /api/cdb_rest/gt_change_status/{name}/{status} — the exact endpoint that locks a GT, which is the trigger point for the materialized view refresh in Rule R9

Current Architecture

The full traffic path from client to database, as implemented in the existing Helm charts, is:



The Infrastructure Gap

The Django charts (npdbchart_sphenix, npdbchart_belle2_hsf) already inject three sets of database credentials into every Django pod:

```
# Already in npdbchart_sphenix/templates/django.yaml
- name: POSTGRES_HOST_W     # write primary  → pgbouncer
- name: POSTGRES_HOST_R1    # read replica 1 → pgbouncer
- name: POSTGRES_HOST_R2    # read replica 2 → pgbouncer
```

And cdb_rest/views.py already implements random read-replica selection:

```
# PayloadIOVsSQLListAPIView in cdb_rest/views.py
read_dbs = [db for db in settings.DATABASES.keys() if db.startswith("read_db_")]
read_db = random.choice(read_dbs) if read_dbs else "default"
```

However, PgBouncer's pgbouncer.ini currently registers only the write primary:

```
# npdbchart_belle2_hsf/templates/pgbouncer.yaml -- current state
[databases]
{{ .Values.dbname_w }} = host={{ .Values.dbhost_w }} port=5432 dbname={{ .Values.dbname_w }}
```

No replica database entries exist, and no PostgreSQL replica StatefulSet is provisioned anywhere. The routing code is fully written and waiting — the infrastructure is not there yet.

Key Codebase Files

Files	Role
cdb_rest/queries.py	The critical LATERAL JOIN SQL — primary optimization target
cdb_rest/models.py	PayloadIOVsSQLListAPIView — read-replica fan-out logic already present
cdb_rest/views.py	PayloadIOV.covering_idx — composite index on (payload_list_id, comb_iov DESC NULLS LAST)
nopayloaddb/db_router.py	Multi-DB router; routes reads to read_db_* keys in DATABASES
nopayloaddb-charts/nopayloaddb/templates/pgbouncer.yaml	PgBouncer ConfigMap — needs replica [databases] entries added
nopayloaddb-charts/npdbchart_sphenix/templates/django.yaml	Django Deployment — POSTGRES_HOST_W/_R1/_R2 already defined
nopayloaddb-charts/npdbchart_belle2_hsf/templates/django.yaml	Promtail sidecar already embedded; Django + Promtail co-located
nopayloaddb-charts/openshift/deployment_psql_server.yaml	Current single-instance PostgreSQL Deployment — to be upgraded
nopayloaddb-charts/openshift/promtail-config.yaml	Existing Promtail config → Loki pipeline
nopayloaddb-charts/nopayloaddb/templates/cron.yaml	CronJob pattern for maintenance tasks (daily pg_dump)
docker-compose-ai.yml	Precedent for an ai_service container alongside the database

3. Problem to be Addressed

3.1 Replica Infrastructure Is Missing Despite the Routing Code Being Ready

The most immediately actionable gap is that the application-side routing for read replicas is fully implemented but the replicas themselves are never provisioned. PayloadIOVsSQLListAPIView contains logic that falls back to the "default" database when no read_db_* keys are present — meaning every read query silently hits the single write primary, regardless of load. PgBouncer's pgbouncer.ini has no [databases] entry for dbname_r1 or dbname_r2, and no PostgreSQL replica StatefulSet exists in any chart.

Furthermore, the existing PostgreSQL server is defined as a plain Deployment (deployment_psql_server.yaml), not a StatefulSet. A Deployment does not guarantee stable network identity or ordered rolling updates — both are required for a primary/replica PostgreSQL topology.

3.2 No Query-Level Observability

There is no pg_stat_statements extension [4] enabled, no Prometheus scraping of query metrics, and no Grafana dashboard for the database tier. The Promtail/Loki pipeline already ships Django/gunicorn logs (as seen in npdbchart_belle2_hsf/templates/django.yaml and openshift/promtail-config.yaml), but PostgreSQL slow query logs are not yet part of it. Operators have no visibility into which queries are slow, whether the covering_idx index is being used by the planner, or whether work_mem is causing sort spills to disk.

3.3 The Core Query Has Non-Trivial Performance Characteristics

The system provides three independent implementations for performing IOV lookups. The two ORM-based approaches (payloadiovs_orm_max/ and payloadiovs_orm_orderby/) do not scale efficiently. In particular, the Max-based implementation executes a separate query for each payload_list_id, resulting in an N+1 query pattern as the number of payload types increases.

The primary production endpoint relies on a SQL-based implementation. Its query, defined in cdb_rest/queries.py, performs a decimal range comparison using a high-precision composite IOV column (comb_iov), represented as a DECIMAL (38,19) value.

```
SELECT pt.name, pi.payload_url, pi.checksum, pi.size,
       pi.major_iov, pi.minor_iov, pi.major_iov_end, pi.minor_iov_end
FROM "PayloadList" pl
JOIN "GlobalTag" gt
  ON pl.global_tag_id = gt.id AND gt.name = %(my_gt)s
JOIN LATERAL (
   SELECT payload_url, checksum, size,
          major_iov, minor_iov, major_iov_end, minor_iov_end
   FROM   "PayloadIOV" pi
   WHERE  pi.payload_list_id = pl.id
     AND  pi.comb_iov <= CAST(
            %(my_major_iov)s
            + CAST(%(my_minor_iov)s AS DECIMAL(19,0)) / 1E19
          AS DECIMAL(38,19))
   ORDER  BY pi.comb_iov DESC NULLS LAST
   LIMIT  1
) pi ON true
JOIN "PayloadType" pt ON pl.payload_type_id = pt.id;
```

The `covering_idx` composite index on `PayloadIOV` (`payload_list_id`, `comb_iov DESC NULLS LAST`) is already in place and serves as the primary performance enabler for this query.

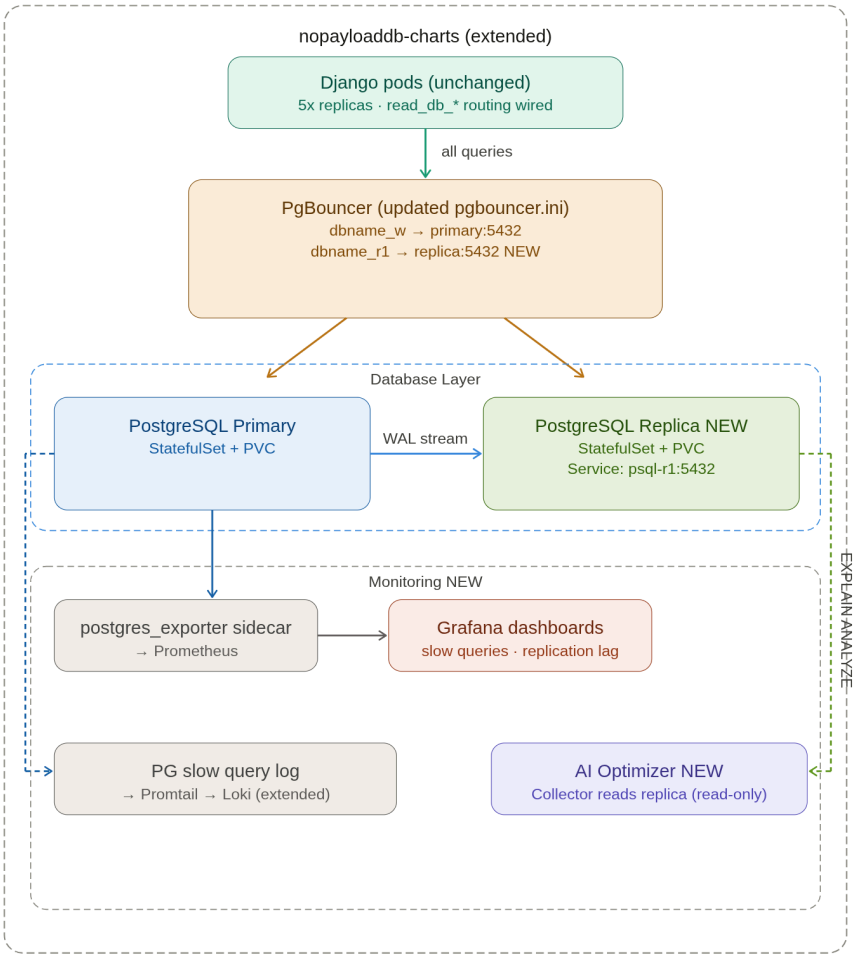
However, its effectiveness depends on several factors: the accuracy of table statistics (which can degrade after bulk `PayloadIOV` inserts following each detector run), the database memory configuration (notably `shared_buffers` and `effective_cache_size`), and the efficiency of autovacuum in controlling index bloat. These factors are not directly observable without collecting and analyzing execution plans (e.g., via `EXPLAIN`) [3] and ongoing performance monitoring.

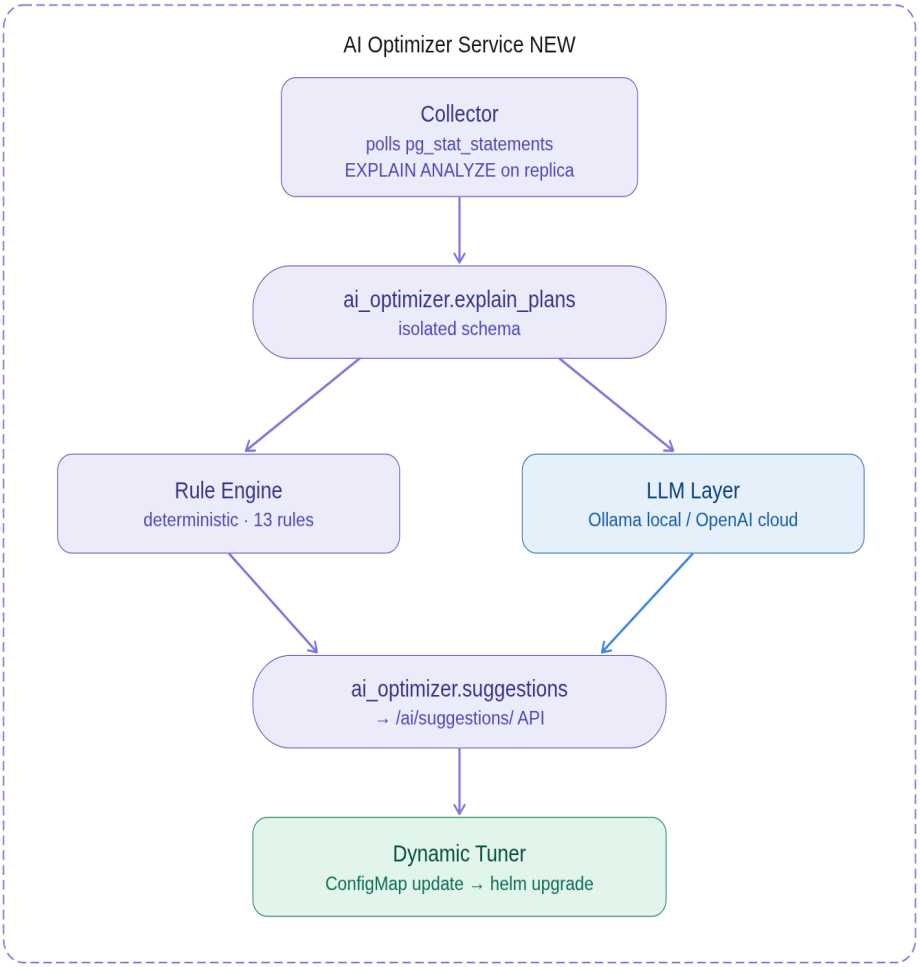
3.4 No Automated Performance Feedback Loop

There is no mechanism to detect when query performance degrades, collect an EXPLAIN ANALYZE plan [3] as evidence, reason about the root cause, and propose a remedy. This applies to both slow queries (high mean_exec_time) and resource-intensive queries that consume excessive I/O or CPU even when they complete within an acceptable wall-clock time — queries with many shared_read_blocks (disk I/O), high rows processed, or frequent lock contention are equally harmful under concurrent experiment workloads but are invisible without metric collection. Database administrators must identify all of these cases reactively and manually. This is unsustainable as sPHENIX accumulates more calibration data over its run lifetime.

4. Proposed Solution

The proposed solution is a four-part, non-invasive framework that adds infrastructure and tooling around the existing system without touching any existing table, migration, Helm template, or application code:





1 — Infrastructure: Convert the existing PostgreSQL Deployment to a StatefulSet, add a streaming replica StatefulSet with PVC, and update the PgBouncer pgbouncer.ini ConfigMap with dbname_r1 entries pointing at the replica — activating the routing logic that already exists in the Django layer.

2 — Observability: Add a postgres_exporter sidecar to the PostgreSQL pods for Prometheus scraping; extend the existing Promtail pipeline to capture PostgreSQL slow query logs; provision Grafana dashboards for the database tier.

3 — AI Analysis Engine: A Python service (following the docker-compose-ai.yml precedent) that automatically collects EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON) plans from the replica for slow queries [4], parses the plan tree, applies deterministic rules, and calls a language model for complex cases — producing structured, operator-reviewable suggestions stored in an isolated ai_optimizer schema.

4 — Dynamic Tuning: AI-driven adjustment of PostgreSQL cluster parameters via ConfigMap updates, with a mandatory operator-approval step before any change takes effect.

Non-invasive by design: No existing table, Django migration, Helm template, or application code is modified. Every artifact is additive — new StatefulSet templates, new ConfigMap entries, a new PostgreSQL schema, a new Python service.

5. Projects Goals

#	Objectives
01	Convert the existing PostgreSQL Deployment to a StatefulSet with PVC and add a streaming read replica StatefulSet, making the existing POSTGRES_HOST_R1 routing live
02	Update PgBouncer's pgbouncer.ini to register replica database entries, completing the connection path from Django through PgBouncer to the replica
03	Establish comprehensive query-level and cluster-level monitoring via pg_stat_statements [4], Prometheus, and Grafana, feeding into the existing Promtail/Loki pipeline
04	Build an automated EXPLAIN plan collection pipeline [3] that fires when queries exceed a configurable latency threshold, executing safely on the replica in read-only transactions
05	Build a layered AI analysis engine (deterministic rules first, LLM second) that identifies bottlenecks and produces safe, structured, operator-reviewable optimization suggestions
06	Implement a dynamic cluster parameter tuner guided by AI insights, with a mandatory approval workflow before any parameter change is applied
07	Validate all recommendations in a staging environment and demonstrate measurable query latency reduction without modifying any existing schemas

6. Implementation Plan

First Half — Infrastructure, Observability, and EXPLAIN Collection

The first half produces the foundation on which the AI engine depends. Every subsequent analysis is only as good as the data it receives, so this half is held to a strict quality bar: no AI work begins until the EXPLAIN plans being collected are demonstrably complete, accurate, and reproducible.

Phase 1: PostgreSQL Cluster and Replica Infrastructure

The first step is to convert the existing PostgreSQL Deployment (`openshift/deployment_psql_server.yaml`) into a StatefulSet. A standard Deployment lacks key capabilities—stable network identity, ordered rolling updates, and proper PVC lifecycle management—all of which are essential for a primary/replica setup.

The new StatefulSet uses a `volumeClaimTemplate`, ensuring each pod gets its own persistent volume. The storage class is passed as a Helm value with no default, requiring the deployer to explicitly choose the appropriate option for their cluster. The pod spec preserves the current security posture with `securityContext.runAsNonRoot: true` and `seccompProfile.type: RuntimeDefault`, in line with OKD's Security Context Constraints, and retains the existing resource configuration (2 CPU / 4 Gi requested, 4 CPU / 8 Gi limit).

Once the primary is stable, a streaming read replica is introduced as a second StatefulSet. A `pg_basebackup` init container initializes the replica's data directory from the primary before startup. It writes `standby.signal` and mounts a recovery configuration (via ConfigMap) with `primary_conninfo` pointing to the primary's headless service DNS. The replica is exposed internally through a dedicated ClusterIP service (`psql-replica:5432`), and replication health can be verified immediately using `pg_stat_replication` on the primary.

With both instances running, the most impactful change in the project is introduced: updating PgBouncer's `pgbouncer.ini` ConfigMap (`npdbchart_belle2_hsf/templates/pgbouncer.yaml`) to include a new `[databases]` entry that maps `dbname_r1` to the replica's service. This small, two-line change activates the `POSTGRES_HOST_R1` routing that Django has already been configured to use—requiring no code changes, migrations, or schema updates.

Finally, the `PayloadIOVsSQLListAPIView` read fan-out to the replica is verified end-to-end using the existing test infrastructure.

Phase 2: Monitoring and Observability Stack

With the cluster topology in place, the next priority is making it fully observable. The pg_stat_statements extension [4] is enabled on the primary by adding a shared_preload_libraries entry to postgresql.conf via init container, with pg_stat_statements.track = all to ensure queries issued through PgBouncer's connection pool are tracked correctly. Alongside this, log_min_duration_statement = 500 is set to start emitting structured slow query logs.

A postgres_exporter sidecar is added to both StatefulSet pod templates, exposing /metrics on port 9187 with collectors for pg_stat_statements, pg_stat_bgwriter, pg_stat_replication, pg_stat_user_tables, and pg_locks. A Prometheus ServiceMonitor registers both pods as scrape targets. Grafana is then provisioned with four dashboards:

- **Slow and Resource-Intensive Query Top-10** — bar chart ranked by a combined score of mean_exec_time and total_blks_read, capturing both slow and I/O-heavy queries from pg_stat_statements.

- **Buffer Health** — shared_blks_hit / (shared_blks_hit + shared_blks_read) ratio over time; alert threshold at 95%.

- **Cluster Health** — replication lag from pg_stat_replication.write_lag, active lock waits from pg_locks, checkpoint frequency from pg_stat_bgwriter, and connection pool utilization from PgBouncer stats.

- **PayloadIOV Vacuum Freshness** — n_dead_tup and last_autovacuum for the PayloadIOV table specifically, since bulk post-run inserts are the main vacuum pressure point.

The existing Promtail configuration (openshift/promtail-config.yaml) already ships Django/gunicorn logs to Loki. A new scrape_configs stanza is added to also capture PostgreSQL slow query logs, and a pipeline_stages regex extracts duration_ms as a structured Loki label. The result is two independent views of slow queries: aggregate statistics from Prometheus and individual query instances from Loki.

Phase 3: EXPLAIN Plan Collection Pipeline

The EXPLAIN collection pipeline bridges raw observability and actionable analysis, built around one hard constraint: it must never touch the production primary. Every EXPLAIN ANALYZE execution runs inside a BEGIN READ ONLY; SET statement_timeout = '5s' transaction against the streaming read replica and is always rolled back — no writes, no side effects, no lock contention on the primary.

The collector service (ai_optimizer/collector/collector.py) runs as a standalone Python process, following the docker-compose-ai.yml companion service precedent. Every 60 seconds it queries pg_stat_statements on the primary for fingerprints exceeding mean_exec_time > 200 ms OR shared_blks_read > 10,000, capturing both slow and resource-intensive queries. Crucially, it is seeded with five high-priority fingerprints from day one, monitored regardless of whether they have yet crossed the threshold:

Priority	Query	Endpoint
1	LATERAL JOIN SQL	GET /api/cdb_rest/payloadiovs/
2	DISTINCT ON ORM	GET /api/cdb_rest/payloadiovs_orm_orderby/
3	Max + Q ORM	GET /api/cdb_rest/payloadiovs_orm_max/
4	Bulk insert	POST /api/cdb_rest/bulk_piov
5	GT clone transaction	POST /api/cdb_rest/cloneGlobalTag/

For each identified fingerprint, the runner opens a BEGIN READ ONLY transaction on the replica, executes EXPLAIN (ANALYZE, BUFFERS, COSTS, VERBOSE, FORMAT JSON) with a 5-second timeout, captures the JSON result, and immediately rolls back. Plans are persisted to ai_optimizer.explain_plans (query hash, query text, DB version, timestamp) — a schema created by the collector's own SQL init scripts, entirely isolated from cdb_rest and never touched by Django migrations. A GIN index on the plan_json column enables efficient JSONB pattern queries across historical plans.

Test coverage operates at two levels: unit tests use pre-recorded EXPLAIN JSON fixtures from the staging cluster (no live database needed), while integration tests run the full end-to-end flow on Minikube — synthetic slow query → pg_stat_statements detection → replica EXPLAIN → DB persistence → Prometheus alert trigger.

Second Half — AI-Assisted Analysis Engine

The second half builds the analysis and recommendation system on top of the observability foundation. The guiding principle is that the AI engine is a decision-support tool, never an autonomous actor. Every suggestion it produces requires explicit human approval before any action is taken.

Phase 4: Plan Parser and Rule-Based Suggestion Engine

The analysis engine begins with a plan parser that recursively walks the PostgreSQL JSON plan tree, converting every node into a typed PlanNode dataclass. Each node captures node_type, relation, actual_rows, plan_rows (the planner's estimate), actual_time_ms, shared_hit_blocks, shared_read_blocks, and a raw properties dict for all other fields. The recursive structure preserves parent-child relationships, which is essential for rules that reason about the interaction between a join node and its inner relation.

The rule engine walks the **parsed tree and applies 13 deterministic rules** split into two groups: five general PostgreSQL pathologies (large sequential scans, stale statistics, hash join spills, external merge sorts, high cache miss ratios) that fire on any schema, and eight codebase-specific rules derived from hands-on EXPLAIN analysis of the actual query plans. The full trigger conditions, categories, and generated SQL for all **13 rules are specified in Section 7.4.**

Each rule produces a Suggestion with category, priority, message, optional safe_sql, and confidence. All safe_sql values are validated against a strict allow-list regex before being stored, so no unsafe command can ever reach the database. The rule engine is the cheapest and most trustworthy layer: zero-latency, fully auditable, no external dependencies.

Phase 5: LLM Integration, Dynamic Tuner, and Suggestion API

For query plans that pass all thirteen rules without producing a suggestion but still show elevated latency or high resource consumption, the engine escalates to the LLM layer. The LLM backend is designed as an injected dependency conforming to a BaseLLMBackend protocol, so Ollama (for BNL air-gapped deployments), OpenAI (for cloud deployments), and MockLLMBackend (for unit tests) are fully interchangeable without changing any application code.

The system prompt is carefully crafted to include the complete schema context — the five-table data model, the covering_idx definition, the approximate row counts, and the specific constraint that no schema-altering commands are acceptable — and uses OpenAI's function-calling feature (or Ollama's structured output mode) to enforce a machine-parseable JSON response format. Every safe_sql value returned by the LLM passes through the same allow-list regex that the rule engine uses before it is ever stored.

The dynamic parameter tuner reads the accumulated suggestion history and pg_stat_statements trends to compute recommended values for the four cluster parameters that most affect this workload:

- **shared_buffers** — increased if buffer hit ratio stays below 95% for 3 consecutive cycles
- **work_mem** — increased if R3/R4 (spill rules) fire more than 5 times in a week
- **autovacuum_vacuum_scale_factor** — reduced to 0.05 for PayloadIOV if R11 fires
- **random_page_cost** — set to 1.1 for Ceph-backed OKD PVs (reflects SSD latency profile)

Approved parameter changes are written to the psql-conf ConfigMap and applied via helm upgrade, which triggers a rolling restart — replica first, then primary — preserving availability throughout.

The suggestion REST API (`/ai/suggestions/`) follows the same Django REST Framework patterns already used in `cdb_rest/`. It provides:

- `GET` to list suggestions, with support for filtering by status and category
- `GET /{id}/` to retrieve full details, including the annotated plan tree
- `PATCH /{id}/` to update the suggestion's status

The system uses a simple state machine — **pending** → **approved** → **applied** — to ensure that every change is explicitly reviewed and authorized.

Once approved:

- `SET` parameter changes are applied immediately
- DDL operations (such as `CREATE INDEX CONCURRENTLY`) are queued as CronJob entries, following the `nopayloaddb/templates/cron.yaml` pattern

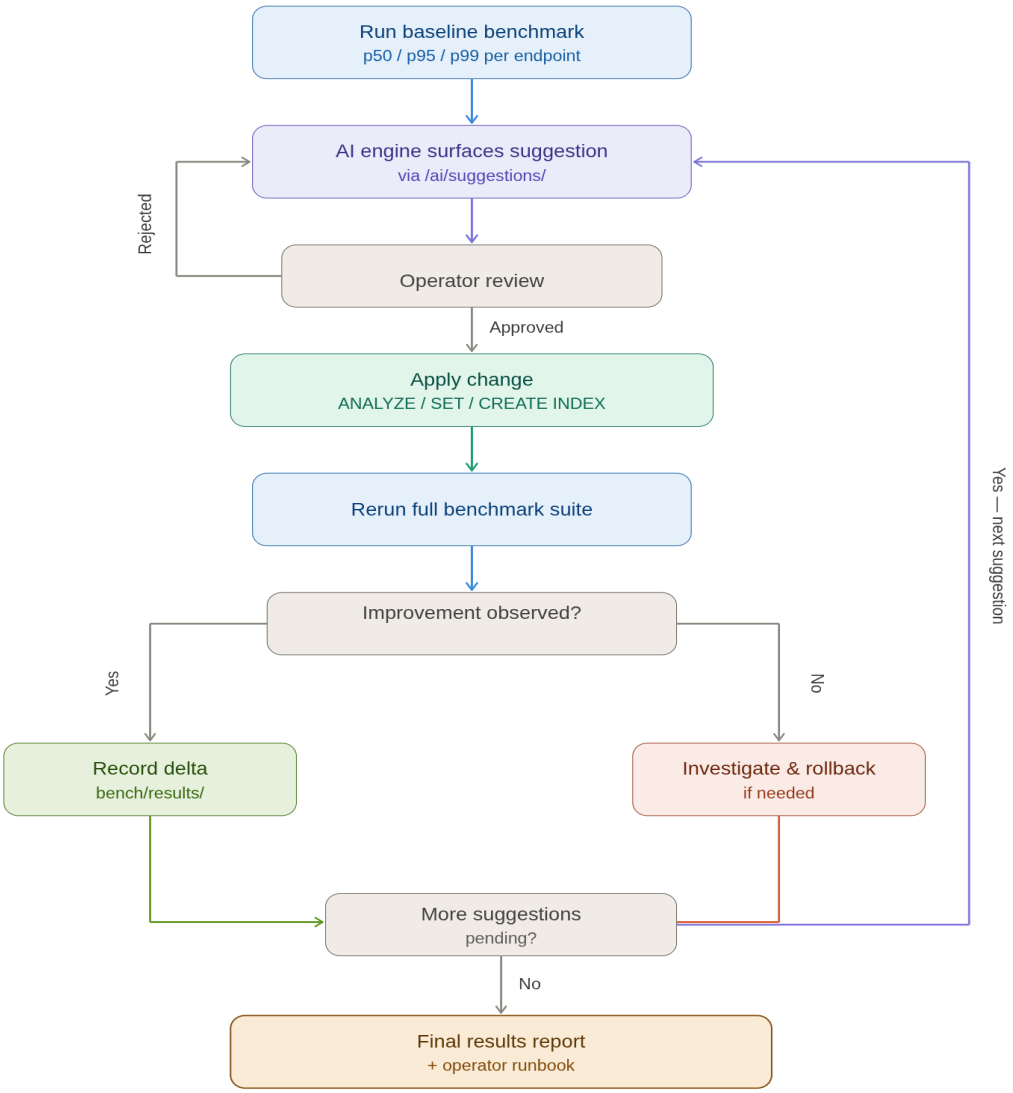
These jobs are then executed during low-traffic, overnight windows to avoid interfering with peak read workloads.

Phase 6: Staging Validation and Documentation

The validation phase begins with building a benchmarking harness (`bench/`) to simulate the full workload. This includes concurrent reads across all three `payloadiovs` endpoint variants, bulk inserts via `POST /api/cdb_rest/bulk_piov`, and GlobalTag cloning through `POST /api/cdb_rest/cloneGlobalTag/`, all running under 50 concurrent clients.

The baseline run captures key performance metrics, including p50, p95, and p99 latency for each endpoint, the buffer hit ratio from Prometheus, and aggregate execution times from `pg_stat_statements` [4] for the five seeded query fingerprints.

These measurements serve as the ground truth, providing a consistent baseline against which every optimization recommendation is evaluated.



AI recommendations are applied incrementally, one at a time. The process starts with high-confidence, low-risk changes—such as autovacuum parameter tuning and `ANALYZE` runs—and gradually moves toward more structural optimizations like extending covering indexes.

After each change, the full benchmark suite is rerun and the performance delta is recorded. This step-by-step approach helps isolate the impact of each recommendation, builds a clear, evidence-based record of what works, and ensures that the combined effect of all approved changes is truly additive rather than conflicting.

Documentation is developed in parallel during the final weeks of benchmarking. The operator runbook (`docs/ai_optimizer.rst`) focuses on three key operational scenarios:

- Deploying a new PostgreSQL StatefulSet cluster from scratch using the updated Helm chart
- Understanding and interpreting the four Grafana dashboards
- Reviewing, approving, and monitoring AI-generated suggestions via the `/ai/suggestions/` API

The runbook is designed for database administrators who are comfortable with PostgreSQL and Kubernetes, but may not be familiar with the internal implementation of the AI engine.

All Helm chart updates are consolidated into a feature branch, with each pull request reviewed and approved. A final release is tagged ahead of the GSoC evaluation deadline to ensure everything is production-ready.

6.1 Project Methodology

The approach is strictly additive—nothing in the existing system is modified. The constraint of not changing any existing tables or schemas is enforced by design. All new database objects are created within a separate `ai_optimizer` PostgreSQL schema. The EXPLAIN collector runs on the read replica using `BEGIN READ ONLY` transactions, ensuring it has no impact on production writes.

On the infrastructure side, all Helm changes are introduced as new resources or clearly defined additions (such as in `pgbouncer.ini`). Existing chart templates remain untouched throughout.

This proposal builds on what already works. Rather than introducing entirely new patterns, it reuses existing infrastructure wherever possible—the Promtail sidecar in `npdbchart_belle2_hsf` serves as the model for extending the log pipeline, the CronJob in `nopayloaddb/templates/cron.yaml` guides how AI tuner runs are scheduled, and `docker-compose-ai.yml` provides a precedent for the AI service container.

As a result, each new component feels like a natural extension of the current system, not a foreign addition.

The system is designed to keep the operator in control. No parameter change or DDL statement is applied automatically—every AI-generated suggestion moves through a **pending** → **approved** → **applied** state machine. This is especially important in a scientific computing environment, where database changes must be auditable and reproducible across production, test, and development setups.

The approach is also test-driven and grounded in real data. EXPLAIN plan JSON fixtures, captured from the staging cluster, are used as deterministic inputs for unit testing the rule engine. The LLM backend is treated as a pluggable dependency, with unit tests relying on a `MockLLMBackend` that returns predefined responses. For end-to-end validation, integration tests are executed on a local Minikube cluster.

Open development. All design decisions are documented in GitHub Issues before any code is written. PRs are opened as drafts within the first two days of each phase.

7. Technical Elements

7.1 PostgreSQL StatefulSet and Replica

Converting the primary : The current `deployment_psql_server.yaml`, which uses a standard Deployment, will be migrated to a StatefulSet. This change introduces a `volumeClaimTemplate`, ensuring stable pod identities, ordered rolling updates, and proper lifecycle management of persistent volumes (PVCs).

```
# nopayloaddb-charts/openshift/statefulset_psql_primary.yaml (new)

apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: psql-primary
  namespace: {{ .Values.project }}
spec:
  serviceName: psql-primary
  replicas: 1
  selector:
    matchLabels:
      app: psql-primary
  template:
    metadata:
      labels:
        app: psql-primary
    spec:
      securityContext:
        runAsNonRoot: true          # required by OKD SCC
        seccompProfile:
          type: RuntimeDefault
      initContainers:
        - name: init-pg-stat-statements
          image: registry.redhat.io/rhel9/postgresql-15
          command: ["sh", "-c",
            "echo \"shared_preload_libraries = 'pg_stat_statements'\" >>
/var/lib/pgsql/data/postgresql.conf &&
             echo \"pg_stat_statements.track = all\" >> /var/lib/pgsql/data/postgresql.conf &&
             echo \"log_min_duration_statement = 500\" >> /var/lib/pgsql/data/postgresql.conf"]
          volumeMounts:
            - name: postgres-data
              mountPath: /var/lib/pgsql/data
      containers:
        - name: postgres
          image: registry.redhat.io/rhel9/postgresql-15
          ports:
            - containerPort: 5432
          resources:
            requests:
              cpu: "2"
              memory: "4Gi"
            limits:
              cpu: "4"
```

```
            memory: "8Gi"
        env:
          - name: POSTGRESQL_DATABASE
            value: {{ .Values.dbname_w }}
          - name: POSTGRESQL_USER
            valueFrom:
              secretKeyRef:
                name: psql-credentials
                key: username
          - name: POSTGRESQL_PASSWORD
            valueFrom:
              secretKeyRef:
                name: psql-credentials
                key: password
        volumeMounts:
          - name: postgres-data
            mountPath: /var/lib/pgsql/data
          - name: psql-conf
            mountPath: /etc/postgresql/conf.d
            readOnly: true
      - name: postgres-exporter         # monitoring sidecar
        image: quay.io/prometheuscommunity/postgres-exporter:v0.15.0
        env:
          - name: DATA_SOURCE_NAME
            valueFrom:
              secretKeyRef:
                name: psql-exporter-dsn
                key: dsn
        ports:
          - containerPort: 9187
            name: metrics
volumeClaimTemplates:
  - metadata:
      name: postgres-data
    spec:
      accessModes: ["ReadWriteOnce"]
      storageClassName: {{ .Values.storageClass }}   # no default -- deployer must set
      resources:
        requests:
          storage: {{ .Values.primaryStorage | default "50Gi" }}
```

Replica StatefulSet. The replica is seeded from the primary using pg_basebackup in an init container:

```
# nopayloaddb-charts/openshift/statefulset_psql_replica.yaml (new)
initContainers:
  - name: seed-from-primary
    image: registry.redhat.io/rhel9/postgresql-15
    command:
      - sh
      - -c
      - |
        pg_basebackup \
          -h psql-primary.{{ .Values.project }}.svc.cluster.local \
          -U replicator \
          -D /var/lib/pgsql/data \
          --wal-method=stream \
          --checkpoint=fast
        touch /var/lib/pgsql/data/standby.signal
    volumeMounts:
      - name: postgres-data
        mountPath: /var/lib/pgsql/data
```

PgBouncer update : One of the most impactful changes to the existing charts is the addition of replica database entries in the `pgbouncer.ini` ConfigMap `npdbchart_belle2_hsf/templates/pgbouncer.yaml`. This allows PgBouncer to route connections appropriately, enabling better utilization of read replicas and improving overall load distribution.

```
[databases]
pgbouncer =
{{ .Values.dbname_w }}  = host={{ .Values.dbhost_w }}  port=5432 dbname={{ .Values.dbname_w }}
{{ .Values.dbname_r1 }} = host=psql-replica.{{ .Values.project }}.svc.cluster.local \
                          port=5432 dbname={{ .Values.dbname_r1 }}
```

This small change—just two lines in `pgbouncer.ini` activates the `POSTGRES_HOST_R1` routing that Django is already configured to use. It requires no changes to Django code, no database migrations, and no schema modifications.

7.2 Monitoring Stack

The monitoring design builds on the existing infrastructure instead of introducing a separate system. `pg_stat_statements` [4] is enabled through an init container that writes to `postgresql.conf`, allowing every SQL query to be tracked using a normalized fingerprint. This provides cumulative metrics such as `total_exec_time`, `calls`, `mean_exec_time`, `stddev_exec_time`, and buffer hit/miss counts.

A `postgres_exporter` sidecar is added to both StatefulSet pods, exposing a `/metrics` endpoint on port 9187. Prometheus scrapes these metrics via a ServiceMonitor, covering key views like `pg_stat_statements`, `pg_stat_bgwriter`, `pg_stat_replication`, `pg_stat_user_tables`, and `pg_locks`.

The existing `openshift/promtail-config.yaml` already forwards Django/Gunicorn logs to Loki. This setup is extended with an additional `scrape_configs` entry to also capture PostgreSQL logs generated by `log_min_duration_statement`.

```
# Extended openshift/promtail-config.yaml
scrape_configs:
  - job_name: django-logs        # existing
    static_configs:
      - targets: [localhost]
        labels:
          job: django
          __path__: /var/log/gunicorn.log
  - job_name: postgres-slow-queries   # NEW
    static_configs:
      - targets: [localhost]
        labels:
          job: postgres
          __path__: /var/lib/pgsql/data/log/postgresql-*.log
    pipeline_stages:
      - match:
          selector: '{job="postgres"}'
          stages:
            - regex:
                expression: 'duration: (?P<duration_ms>[\d.]+) ms'
            - labels:
                duration_ms:
```

Grafana dashboards — four JSON dashboard definitions provisioned at deploy time:

1. Slow and Resource-Intensive Query Top-10 — bar chart ranked by a combined mean_exec_time + total_blks_read score, capturing both slow and I/O-heavy queries.

2. Buffer Health — shared_blks_hit / (shared_blks_hit + shared_blks_read) over time; alert at 95%.

3. Cluster Health — replication lag (pg_stat_replication.write_lag), lock waits (pg_locks), checkpoint frequency (pg_stat_bgwriter), and PgBouncer connection utilization.

4. PayloadIOV Vacuum Freshness — n_dead_tup and last_autovacuum for the PayloadIOV table specifically, since bulk post-run inserts are the main vacuum pressure point.

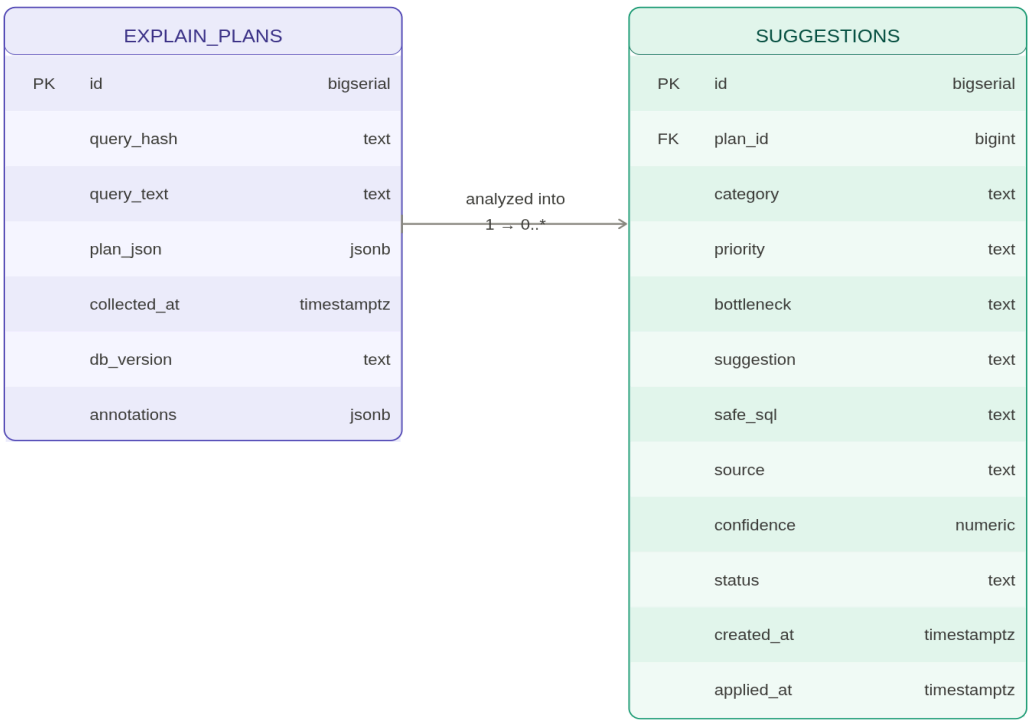
7.3 EXPLAIN Plan Collection Pipeline

The collector identifies slow and resource-intensive queries from pg_stat_statements [4] and captures their plans from the replica using Explain [3]. The five seeded high-priority fingerprints are listed in Phase 3 of the Implementation Plan above. The core collection logic is:

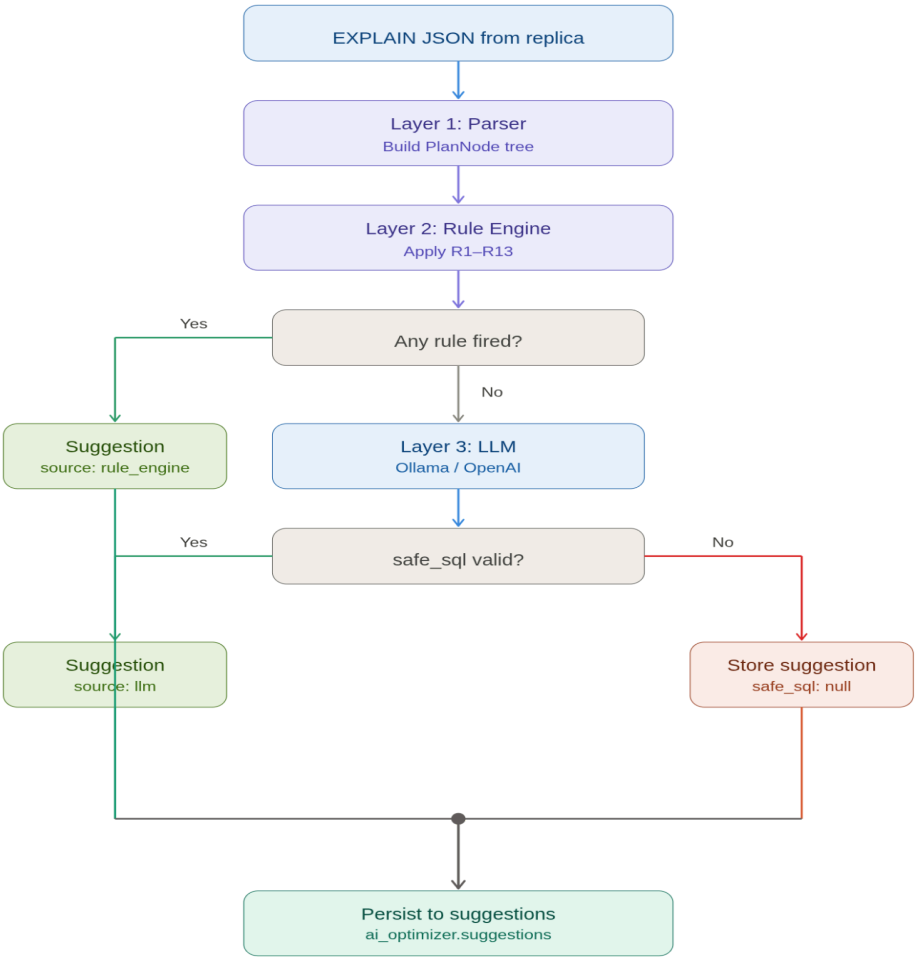
```
# ai_optimizer/collector/collector.py
SLOW_QUERY_THRESHOLD_MS  = float(os.getenv("SLOW_QUERY_THRESHOLD_MS", "200"))
IO_INTENSIVE_THRESHOLD   = int(os.getenv("IO_INTENSIVE_THRESHOLD", "10000"))  # blks read

def collect_once(primary_conn, replica_conn):
    with primary_conn.cursor() as cur:
```

```
        # Capture both slow queries (high mean_exec_time) AND
        # resource-intensive queries (high shared_blks_read = disk I/O pressure)
        cur.execute("""
            SELECT queryid, query, mean_exec_time, calls, rows,
                   shared_blks_read, shared_blks_hit,
                   total_exec_time / NULLIF(calls, 0) AS mean_ms
            FROM   pg_stat_statements
            WHERE  mean_exec_time > %s
               OR  shared_blks_read > %s
            ORDER  BY (mean_exec_time * calls + shared_blks_read) DESC
            LIMIT  20
        """, (SLOW_QUERY_THRESHOLD_MS, IO_INTENSIVE_THRESHOLD))
        flagged_queries = cur.fetchall()

    for queryid, query_text, mean_ms, calls, rows, blks_read, blks_hit, _ in flagged_queries:
        explain_json = run_explain_on_replica(replica_conn, query_text)
        if explain_json:
            store_plan(primary_conn, queryid, query_text, explain_json)

def run_explain_on_replica(conn, query_text: str) -> dict | None:
    """Runs EXPLAIN ANALYZE in a read-only transaction -- never executes writes."""
    try:
        with conn.transaction():
            conn.execute("SET TRANSACTION READ ONLY")
            conn.execute("SET statement_timeout = '5s'")
            result = conn.execute(
                f"EXPLAIN (ANALYZE, BUFFERS, COSTS, VERBOSE, FORMAT JSON) {query_text}"
            ).fetchone()
            raise psycopg.errors.QueryCanceled  # rollback -- we never want to commit
    except psycopg.errors.QueryCanceled:
        return result[0] if result else None
```

All plans are stored in a schema that is completely isolated from the cdb_rest application schema:



7.4 AI Analysis Engine



The engine is structured in three independent layers, executed sequentially—starting with lightweight, deterministic checks and only invoking more expensive model calls when necessary.

The first layer, the plan parser, recursively traverses the PostgreSQL JSON execution plan and converts it into a typed `PlanNode` dataclass hierarchy. In doing so, it extracts the key signals required by the rule engine.

```
@dataclass
class PlanNode:
    node_type: str            # "Seq Scan", "Index Scan", "Hash Join", ...
    relation: str | None      # table name if applicable
    startup_cost: float
    total_cost: float
    plan_rows: int            # planner's estimate
    actual_rows: int          # real count from ANALYZE
    actual_time_ms: float
    shared_hit_blocks: int    # served from shared_buffers
    shared_read_blocks: int   # read from disk (cache miss)
    children: list["PlanNode"]
    properties: dict          # all remaining plan keys
```

Layer 2 — Rule-Based Suggestion Engine

This layer runs before any LLM is invoked. It consists of thirteen rules, each designed to detect a specific, real-world performance issue commonly seen in conditions database workloads [2].

Notably, Rules R6 through R13 derived directly from hands-on analysis of EXPLAIN plans [3] for the payloadiovs/ endpoint during the pre-proposal phase, grounding the system in observed behavior rather than assumptions

Rule	Trigger	Category	Suggestion
R1: Large Seq Scan	node_type == "Seq Scan" AND actual_rows > 10,000	INDEX	CREATE INDEX CONCURRENTLY
R2: Stale Statistics	actual_rows / plan_rows > 10x or < 0.1x	STATISTICS	ANALYZE "{table}"
R3: Hash Join Spill	node_type == "Hash" AND Hash Batches > 1	WORK_MEM	Increase work_mem
R4: External Sort	Sort Method contains "external merge"	WORK_MEM	Increase work_mem
R5: Cache Miss	shared_read_blocks / total_blocks > 10%	SHARED_BUFFERS	Increase shared_buffers
R6: covering_idx Bypassed	PayloadIOV Seq Scan when covering_idx exists	STATISTICS	ANALYZE "PayloadIOV" — stale stats after bulk insert
R7: Heap Fetch on Hot Path	LATERAL subquery uses Index Scan (not Index Only Scan) on PayloadIOV	INDEX_COVERAGE	Extend covering_idx with INCLUDE clause to enable index-only scans
R8: Repeated Identical Queries	pg_stat_statements.calls > 1000 AND stddev_exec_time < 5ms for same queryid	CACHE	Suggest application-layer Redis cache keyed on (gtName, majorIOV, minorIOV)
R9: Locked GT High Read Volume	High calls on payloadiovs/ for a GT with status='locked'; trigger: PUT /api/cdb_rest/gt_change_status/{name}/locked	MATERIALIZED_VIEW	Pre-compute latest IOV per PayloadType as a MATERIALIZED VIEW; refresh at lock time
R10: Nested Loop Fanout	Nested Loop with actual_rows > 100 per outer row	INDEX	Suggest join-order adjustment or index on inner relation
R11: Autovacuum Lag	n_dead_tup > 5% of n_live_tup on PayloadIOV; correlates with POST /api/cdb_rest/bulk_piov call volume	VACUUM	Lower autovacuum_vacuum_scale_factor to 0.05 for PayloadIOV
R12: GT Clone Slow Under Load	GlobalTagCloneAPIView @transaction.atomic runtime > 5s on large GTs	CLONE	Suggest off-peak scheduling; flag long-held transaction lock
R13: Repeated Subquery Materialization	EXPLAIN shows the same subquery node executed many times across LATERAL iterations	TEMP_TABLE	Suggest materializing the intermediate result into a temporary table

R6 and R7 focus on the two most critical index-related issues in this schema.

- R6 detects stale statistics, which are the most common reason a covering index gets ignored—especially after bulk sPHENIX runs inserts via `POST /api/cdb_rest/bulk_piov`.

- R7, on the other hand, identifies a structural gap in the index. In this case, the LATERAL subquery needs to fetch fields like `payload_url`, `checksum`, `size`, `major_iov`, `minor_iov`, `major_iov_end`, and `minor_iov_end` directly from the heap, because the current index only includes `(payload_list_id, comb_iov DESC NULLS LAST)`.

This rule is triggered when the query plan shows an `Index Scan` instead of an `Index Only Scan`, indicating that the index is not fully covering the query. When R7 fires, it generates:

```
-- safe_sql generated by Rule R7
CREATE INDEX CONCURRENTLY covering_idx_v2
ON "PayloadIOV" (payload_list_id, comb_iov DESC NULLS LAST)
INCLUDE (payload_url, checksum, size,
         major_iov, minor_iov, major_iov_end, minor_iov_end);
```

The `INCLUDE` clause (PostgreSQL 11+) allows non-key columns to be stored in index leaf pages, enabling index-only scans without affecting sort order. The trade-off—a larger index and slightly slower bulk writes—is made explicit in the suggestion message so operators can weigh the impact.

R8 and R9 go beyond database-level tuning and introduce system-level optimizations.

- **R8** identifies cacheable read patterns based on a clear statistical signal: high call volume (> 1000) with near-zero stddev_exec_time. When triggered, it recommends introducing a Redis cache keyed by (gtName, majorIOV, minorIOV). For locked GlobalTags, where the IOV data is immutable, using a long TTL is completely safe.

- **R9** focuses on simplifying expensive queries. For locked GlobalTags, it proposes precomputing the "latest valid IOV per PayloadType" as a materialized view. This replaces a complex four-table join with decimal range filtering by a single indexed lookup. The `GlobalTagChangeStatusAPIView` lock event serves as the trigger for `REFRESH MATERIALIZED VIEW CONCURRENTLY`, following the existing CronJob pattern in `nopayloaddb/templates/cron.yaml`.

R11 links storage-level signals with workload behavior. It correlates spikes in dead tuples in `PayloadIOV` with increased call counts in `pg_stat_statements` for the bulk insert query fingerprint. This ensures that `ANALYZE` and `VACUUM` recommendations are triggered at the right time, rather than on a fixed schedule.

R12 detects long-running transactions caused by the `@transaction.atomic` decorator in `GlobalTagCloneAPIView`, particularly for large GlobalTags. It recommends shifting these operations to off-peak windows using CronJobs.

R13 addresses repeated subquery evaluation across LATERAL iterations by suggesting materialization of intermediate results. For example:

```
-- safe_sql generated by Rule R13
CREATE TEMP TABLE _gt_lookup ON COMMIT DROP AS
 SELECT id, name, status_id FROM "GlobalTag" WHERE name = %(my_gt)s;

-- rewrite outer query to JOIN against _gt_lookup instead of "GlobalTag"
```

Temporary tables exist only for the duration of a session and are automatically dropped when the connection closes. This means no schema changes and no permanent objects are introduced. Because this optimization requires coordination at the application layer, it is presented as an advisory suggestion rather than an automatically applied change.

As a future stretch goal (post-GSoC), the system will introduce adaptive scaling recommendations. When `pg_stat_user_tables.n_live_tup` for `PayloadIOV` exceeds 100 million rows, the engine will surface a low-priority suggestion to implement hash partitioning.

This approach aligns with the existing covering index key and helps distribute query load more evenly across PostgreSQL workers, improving performance at scale.

Layer 3 — LLM-Augmented Analysis

For plans that pass all rules but still show high latency, the engine calls a language model with structured output enforced via function-calling:

```
System:
  You are a PostgreSQL optimization expert for a HEP conditions database.
  The schema has: GlobalTag → PayloadList → PayloadIOV (covering_idx on
  payload_list_id, comb_iov DESC NULLS LAST). PayloadIOV has ~15M rows.
  NEVER suggest DROP, TRUNCATE, ALTER TABLE, or schema changes.
  Return ONLY valid JSON: {"bottleneck": "...", "suggestion": "...", "safe_sql": "..." | null}

User:
  Query took {actual_ms}ms. EXPLAIN (ANALYZE, BUFFERS) plan:
  {plan_json_flattened}
```

Every safe_sql value is validated against a strict allow-list before being stored:

```
SAFE_SQL_ALLOW = re.compile(
    r"^(CREATE INDEX CONCURRENTLY|ANALYZE|VACUUM|REINDEX CONCURRENTLY"
    r"|SET \w[\w.]*\s*=|ALTER INDEX \w+ RENAME TO"
    r"|CREATE TEMP TABLE \w+ ON COMMIT DROP)\b",
    re.IGNORECASE,
)

def validate_safe_sql(sql: str | None) -> str | None:
    if sql and SAFE_SQL_ALLOW.match(sql.strip()):
        return sql.strip()
    return None  # silently drop anything that doesn't match
```

The LLM backend is an injected dependency (BaseLLMBackend protocol), making local Ollama, OpenAI, and a MockLLMBackend for tests fully interchangeable.

7.5 Dynamic Parameter Tuner

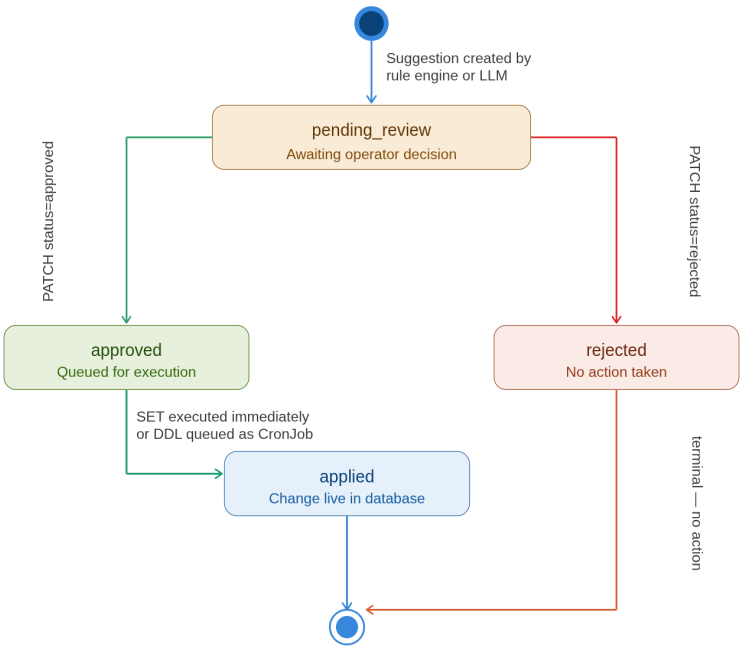
The tuner aggregates suggestion history and pg_stat_statements [4] trends to compute recommended values for the parameters most impactful for this workload:

Parameter	Tuning Logic
shared_buffers	25% of pod memory limit; escalate if buffer hit ratio < 95%
effective_cache_size	75% of node RAM
work_mem	pod_mem_limit / (max_connections × 2); raise if R3/R4 rules fire repeatedly
autovacuum_vacuum_scale_factor	Reduce to 0.05 for PayloadIOV — bulk post-run inserts create high dead-tuple pressureReduce to 0.05 for PayloadIOV — bulk post-run inserts create high dead-tuple pressure
random_page_cost	Set to 1.1 for Ceph-backed OKD PVs (effectively SSD latency)
checkpoint_completion_target	Set to 0.9 to spread checkpoint I/O across the checkpoint window
log_min_duration_statement	Set to 500ms to feed the Promtail/Loki slow-query pipeline

Approved changes are written into the psql-conf ConfigMap mounted by the StatefulSet. A helm upgrade is then triggered, which rolls out the new configuration — zero-downtime for the replica, one brief restart for the primary.

7.6 Suggestion REST API

All suggestions are reviewed through a minimal REST API, built using the same Django REST Framework conventions already established in `cdb_rest/`. The design is intentionally simple: every suggestion moves through a **pending** → **approved** → **applied** state machine, ensuring that no change is applied without explicit authorization.



Once approved, `SET` parameter changes are applied immediately, while DDL operations are queued as CronJob jobs and executed during low-traffic, overnight windows.

The API exposes three endpoints:

```
GET  /ai/suggestions/           list all suggestions (filter: ?status=pending_review)
GET  /ai/suggestions/{id}/      full detail including plan JSON and EXPLAIN tree
PATCH /ai/suggestions/{id}/     {"status": "approved"} -- triggers apply for SET commands
                                {"status": "rejected"} -- no action taken
```

Example suggestion payload:

```
{
  "id": 7,
  "query_hash": "4f2a91bc",
  "collected_at": "2026-07-22T11:03:00Z",
  "bottleneck": "Seq Scan on PayloadIOV (1.4M rows) -- covering_idx bypassed due to stale
statistics",
  "suggestion": "Run ANALYZE on PayloadIOV to refresh row estimates. Planner is currently
estimating 94 rows but seeing 1.4M.",
  "safe_sql": "ANALYZE \"PayloadIOV\";",
  "category": "STATISTICS",
  "priority": "HIGH",
  "source": "rule_engine",
  "confidence": 1.0,
  "status": "pending_review"
}
```

DDL suggestions—such as `CREATE INDEX CONCURRENTLY`, which require longer execution times—are queued as CronJob jobs following the existing `nopayloaddb/templates/cron.yaml` pattern. These jobs are scheduled during low-traffic, overnight windows to avoid competing with peak read workloads.

8. Challenges and Proposed Solutions

Challenge 1: PgBouncer is in the Connection Path — Replica Must Be Registered There

Unlike a direct-connection setup, Django does not connect to PostgreSQL directly. All connections go through PgBouncer. If a replica is provisioned but not registered in pgbouncer.ini, the POSTGRES_HOST_R1 routing is still dead.

Solution: The PgBouncer `pgbouncer.ini` ConfigMap update (D3) is the key enabler and is prioritized for Week 3 of Phase 1. This change introduces an entry such as `dbname_r1 = host=psql-replica... port=5432 dbname=...` in the `[databases]` section, allowing traffic to be routed to the replica.

Because PgBouncer supports hot reloading via the `RELOAD` command on the admin socket, this update can be applied in production without requiring a restart.

Challenge 2: EXPLAIN ANALYZE Executes the Query — Cannot Run on Primary Under Load

EXPLAIN ANALYZE actually runs the query. Running it on the production primary adds real CPU and I/O load.

Solution: The collector operates exclusively on the streaming read replica, running inside a `BEGIN READ ONLY` transaction with `statement_timeout = '5s'`, and is always rolled back. This ensures zero writes and no impact on the primary database.

The `statement_timeout` acts as a safeguard, preventing any runaway queries. This setup aligns precisely with the intended role of the read replica.

Challenge 3: AI-Generated SQL Could Be Unsafe

A language model prompted to suggest optimizations could hallucinate dangerous commands (DROP INDEX, ALTER TABLE, TRUNCATE).

Solution: Two independent guards are in place :

- First, the system prompt strictly limits what the AI can generate. It explicitly forbids any schema-altering commands and clearly defines the set of permitted operations.

- Second, every `safe_sql` output is validated against a strict allow-list using regex. Only approved commands—such as `CREATE INDEX CONCURRENTLY`, `ANALYZE`, `VACUUM`, `SET`, and `REINDEX CONCURRENTLY`—are accepted before being stored or shown to an operator.

  If a suggestion does not match this allow list, it is safely discarded. The suggestion itself is still recorded, but with `safe_sql: null`, ensuring that no unverified command is ever presented for approval.

Challenge 4: OKD Security Context Constraints (SCC)

OKD enforces `runAsNonRoot: true` by default, as seen in `deployment_psql_server.yaml`. The Red Hat PostgreSQL image (`registry.redhat.io/rhel9/postgresql-15`) runs as UID 26 out of the box and is fully compliant with these security constraints.

However, the `pg_basebackup` init container used for the replica must also have the correct permissions on the data volume. In particular, it needs access under the same UID to ensure it can properly initialize and populate the database files.

Solution: Both the primary and replica StatefulSet templates explicitly define `securityContext.runAsNonRoot:true` and `seccompProfile.type: RuntimeDefault`, aligning with the existing deployment's security posture.

The `volumeClaimTemplate` uses `ReadWriteOnce`, which is the appropriate access mode for a single-pod StatefulSet running on Ceph.

To avoid permission issues, the init container runs with the same UID as the main container, ensuring consistent access to the mounted volume.

Challenge 5: covering_idx Degrades After Bulk Inserts

Each sPHENIX detector run triggers large bulk inserts of PayloadIOV rows via PayloadIOVBulkCreationAPIView. After these inserts, autovacuum may not have run yet, statistics become stale, and the planner stops using covering_idx — switching to a Seq Scan and degrading the critical LATERAL JOIN query from milliseconds to seconds.

Solution: Rule R6 checks whether the `PayloadIOV` table is being sequentially scanned despite the presence of a covering index. When this condition is detected, it immediately generates a high-priority `ANALYZE "PayloadIOV"` recommendation.

In addition, the parameter tuner adjusts `autovacuum_vacuum_scale_factor` for `PayloadIOV`, lowering it from the default 0.20 to 0.05. This change causes autovacuum to trigger after just 5% dead-tuple accumulation instead of 20%, helping keep statistics up to date more proactively between runs

Challenge 6: Schema Isolation Without Touching Django Migrations

Introducing any new table into the cdb_rest Django app would require creating Django migrations that could conflict with the shared migration history across BNL production, sPHENIX test, and developer environments.

Solution: All new tables—such as `ai_optimizer.explain_plans` and `ai_optimizer.suggestions`—are created in a dedicated PostgreSQL schema (`ai_optimizer`). They are managed through plain SQL scripts owned by the AI service, rather than Django's migration framework.

This follows the same pattern as `pg_stat_statements`, which manages its own catalog objects independently of application migrations. As a result, the `cdb_rest` Django app's migration history remains completely untouched.

9. Timeline

Timeline	Phase	Milestones
May 1 - May 24 (2026)	**Community Bonding** (Pre-coding Period)	• Discuss project scope, priorities, and communication norms with mentors • Set up Minikube dev environment and confirm CI pipeline • Deploy nopayloaddb locally; run all existing tests to establish a baseline • Capture real EXPLAIN JSON fixtures from the staging cluster for use as unit test inputs • Deep-dive into Helm chart structure • Finalize any open proposal questions and agree on the week-by-week review cadence
Week 1 - Week 4 (May 25 - June 21) (2026)	**Phase 1 — PostgreSQL Cluster & Replica** **Deliverable:** • PostgreSQL primary StatefulSet template with PVC • PostgreSQL replica StatefulSet template with PVC and pg_basebackup init container • Updated PgBouncer pgbouncer.ini with dbname_r1 entry pointing to replica	• Convert deployment_psql_server.yaml to StatefulSet with volumeClaimTemplate • Add securityContext constraints required by OKD SCC • Add streaming replica StatefulSet with pg_basebackup init container • Provision ClusterIP service (psql-replica:5432) and verify pg_stat_replication lag = 0 • Extend PgBouncer pgbouncer.ini with dbname_r1 entry • Verify PayloadIOVsSQLListAPIView read fan-out hitting replica end-to-end
Weeks 5 – Week 7 (June 22 - July 12) (2026)	**Phase 2 — Monitoring & Observability** **Deliverable:** • **Monitoring stack:** postgres_exporter sidecar + Prometheus ServiceMonitor + Grafana dashboards • Promtail config extension: PostgreSQL slow query log → Loki	• Enable pg_stat_statements and log_min_duration_statement via init container • Deploy postgres_exporter sidecar on both pods; register Prometheus ServiceMonitor • Provision all four Grafana dashboards (Slow Query Top-10, Buffer Health, Cluster Health, Vacuum Freshness) • Extend Promtail pipeline for PostgreSQL slow query logs; confirm duration_ms label queryable in Loki

Timeline	Phase	Milestones
(July 6 - July 10) (2026)	🏁 **Midterm Evaluation**	• Collect feedback from mentors on infrastructure and observability results • Confirm replica routing, monitoring stack, and Promtail pipeline are production-ready • Review all first-half PRs with mentors • Adjust scope or approach for the EXPLAIN collection and AI engine phases if needed before starting Phase 3
Weeks 8 – Week 12 (July 13 - August 16) (2026)	**Phase 3 — EXPLAIN Collection Pipeline** **Deliverable:** • EXPLAIN plan collector service (collector.py) • Unit and integration tests (Partial)	• Build collector.py polling pg_stat_statements every 60s • Detect slow and I/O-intensive fingerprints • Implement run_explain_on_replica with BEGIN READ ONLY + forced rollback • Persist plans to ai_optimizer.explain_plans with GIN index on plan_json • Unit tests with captured EXPLAIN JSON fixtures; integration tests on Minikube
Weeks 13 – Week 17 (August 17 - Sept 20) (2026)	**Phase 4 — Plan Parser & Rule Engine** **Deliverable:** • Plan parser + Rule-based suggestion engine • Unit and integration tests (Partial)	• Build recursive PlanNode dataclass parser from PostgreSQL JSON plan tree • Implement R1–R5 (general PostgreSQL pathologies) with fixture-driven unit tests • Implement R6–R13 (codebase-specific rules) covering index gap, Redis cache, materialized view, autovacuum lag, temp table materialization • safe_sql allow-list validation; Suggestion dataclass with category, priority, confidence • Full rule engine test suite with ≥80% coverage

Timeline	Phase	Milestones
Weeks 18 – Week 21 (Sept 21 - October 18) (2026)	**Phase 5 — LLM, Parameters & API** **Deliverable:** • LLM-augmented analysis layer with safe_sql allow-list validation • Dynamic parameter tuner • Suggestion REST API (/ai/suggestions/ GET / PATCH) with approval workflow • Unit and integration tests (Complete)	• BaseLLMBackend protocol with Ollama, OpenAI gpt-4o-mini, and MockLLMBackend • Structured JSON output enforced via function-calling; safe_sql allow-list applied to all LLM output • Parameter recommendation module (shared_buffers, work_mem, autovacuum suggestions) • /ai/suggestions/ REST API (GET / PATCH) with pending → approved → applied state machine • CronJob queue for DDL commands; end-to-end integration tests
Weeks 22 – Week 24 (October 19 - Nov 1) (2026)	**Phase 6 — Validation & Documentation** **Deliverable:** • Benchmarking harness and comparative results report • Documentation and Report	• Benchmarking harness (bench/) under 50 concurrent clients; capture baseline p50/p95/p99 per endpoint • Apply recommendations one-by-one; record deltas in bench/results/ • Write docs for all three operational scenarios • All Helm chart PRs reviewed and merged; release tagged
November 2, 2026	**Final Deadline**	• Final Project Submission

Availability

I'm fully committed to working on the AI-Assisted PostgreSQL Optimization project for GSoC this summer and don't have any other obligations that would interfere with my work. I do have my upcoming 8th semester exams, but they'll wrap up within a week of the coding period starting, so they won't impact my development schedule.

I plan to dedicate around 35–40 hours per week to the project, which is why I've applied for the 350-hour project. If anything unexpected comes up, I'll communicate it early and make up for lost time to stay on track with milestones.

My usual working hours are 9:00–14:00 and 16:30–22:30 IST (UTC+5:30). That said, I'm flexible and can adjust based on mentor availability.

I'll share a written progress update every Monday, aim to have draft PRs ready within the first two days of each phase, and schedule a weekly sync with mentors at a mutually convenient time.

Post-GSoC Plans and Community Engagement

I am confident that all proposed deliverables — the PostgreSQL StatefulSet cluster, monitoring stack, EXPLAIN plan collection pipeline, AI analysis engine, dynamic parameter tuner, and suggestion API — will be implemented within the GSoC timeline. Should any stretch goals remain incomplete, I commit to finishing them during the extended period after the final evaluation.

Having already familiarized myself with the nopayloaddb codebase, the HSF conditions database ecosystem, and the sPHENIX and Belle II deployment patterns, I plan to remain an active contributor beyond GSoC. Specifically, I intend to help integrate the AI optimizer into additional experiment deployments, extend the rule engine with new rules as detector run patterns evolve, and mentor future contributors who want to work on the database or observability layers. The intersection of high-energy physics computing and database performance engineering is an area I find genuinely compelling, and I am enthusiastic about continuing to improve a platform that directly supports real detector calibration workflows at BNL and KEK.

10. Biographical Information

Educational Background

I am currently pursuing a B.Tech in Information Technology at Jaypee Institute of Information Technology, with an expected graduation in 2026. Throughout my academic journey I have focused on backend systems, database engineering, and infrastructure automation — areas that align directly with the technical requirements of this project.

My coursework and personal projects have given me hands-on experience with Python and Django REST Framework, PostgreSQL query analysis and indexing strategies, Kubernetes and Helm chart authoring, and applied AI/ML concepts including structured LLM output and prompt engineering.

Relevant Experience

My 2+ years of hands-on experience in backend and full-stack development using Python, JavaScript, and modern frameworks has equipped me with the skills needed to contribute effectively to this project. Having worked extensively with distributed systems, Kubernetes, and database-driven applications, I am comfortable working across the stack—from infrastructure to application-level optimization.

The architecture of this project, involving PostgreSQL, Django, and Kubernetes-based deployment, aligns closely with my prior experience. I have practical exposure to monitoring, performance tuning, and building scalable systems, which enables me to quickly understand bottlenecks and contribute to optimizing query performance. This background will allow me to actively work on analyzing EXPLAIN plans, integrating monitoring pipelines, and implementing AI-assisted optimization strategies with minimal ramp-up time.

Backend Developer Intern at GrexaAI (1M+ users, distributed microservices)

During my time at GrexaAI, I worked on backend services in a high-scale distributed system, which gave me hands-on experience with performance and reliability challenges in production.

Key Contributions Relevant to the Project:

- I built and maintained high-throughput services using Go, gRPC, Redis, and MongoDB, where handling latency and efficiency was critical

- While designing caching layers with Redis, I used composite keys to structure data efficiently — this directly relates to how I plan to design cache keys like (gtName, majorIOV, minorIOV) in Rule R8

- I also worked on decoupled companion services that observe production systems without interfering with the main request flow, a pattern I plan to reuse for the EXPLAIN collector running on a read replica

- For performance debugging, I regularly used tools like MongoDB query profilers and Redis MONITOR, which has helped me develop an intuition for analyzing database behavior — something I will apply when working with pg_stat_statements and EXPLAIN plans

Linux Foundation Mentorship Intern — KubeEdge (CNCF)

During my mentorship with KubeEdge, I worked on improving how workloads are scheduled and configured in Kubernetes-based environments, particularly in distributed and edge setups. Through this work, I spent significant time working closely with Kubernetes, which helped me develop a deeper understanding of its architecture — especially how scheduling, node affinity, and control plane decisions influence workload placement across distributed systems.

Key Contributions Relevant to the Project:

- This experience with Kubernetes scheduling directly translates to setting up the PostgreSQL cluster using **StatefulSets and node affinity**, where pods need to be placed on specific nodes (e.g., SDCC workers).

- My work on dynamic configuration management is also relevant to implementing **ConfigMap-based PostgreSQL parameter tuning**, where database settings can be updated and applied without redeploying the entire system.

DevSecOps Deploy Netflix Clone

In this project, I designed and deployed a complete CI/CD pipeline for a containerized application on AWS, using Kubernetes and Helm for orchestration and ArgoCD for GitOps-based delivery.

- I built an end-to-end deployment pipeline using Jenkins, Docker, and Kubernetes, ensuring reliable and repeatable releases

- I set up a Prometheus-Grafana monitoring stack to track system performance and observability

- I also integrated Trivy-based vulnerability scanning, identifying and resolving 36+ critical container security issues

Key Contributions Relevant to the Project:

- My experience writing and managing Helm charts will directly help in setting up the PostgreSQL cluster and handling updates like **rolling restarts and configuration changes** through ConfigMaps

- The monitoring stack I implemented closely matches the requirements of this project, including **metric collection, scrape configurations, and dashboard creation** for tracking database and system performance

- Working on container security and deployment constraints has also given me familiarity with Kubernetes security practices, which will be useful when deploying PostgreSQL in environments like OpenShift/OKD with stricter policies

CognitiveWeb — RAG Chatbot RAG

In this project, I built an intelligent chatbot that combines retrieval-augmented generation (RAG) with real-time web data to provide accurate and context-aware responses.

Key Contributions Relevant to the Project:

- My experience with prompt engineering and structuring LLM outputs will help in designing the AI layer to generate **safe, structured, and machine-readable recommendations** (e.g., JSON-based outputs instead of free-form text)

- I have developed an understanding of when to rely on LLMs versus deterministic logic, which aligns with the approach of using **rule-based optimization first, and AI as a fallback for more complex cases**

NutriFY — AI Nutrition Assistant Python

Built multi-agent AI backend coordinating meal planning across 600+ food items with real-time tracking and personalised nutrient guidance using OpenAI GPT-4

Key Contributions Relevant to the Project:

- OpenAI function-calling experience (GPT-4 structured output) is directly applicable to the LLM layer — the proposal uses the same mechanism to enforce {"bottleneck": ..., "suggestion": ..., "safe_sql": ...} JSON response format

- Multi-agent workflow design experience informs the layered engine architecture: parser → rule engine → LLM is a sequential agent pipeline where each layer has a defined contract

Personal Information

Name	Sarthak Negi
Email	sarthaknegi908@gmail.com
Github	github.com/sarthaksarthak9
LinkedIn	linkedin.com/sarthaknegi9
X	x.com/sarthaknegi009
Portfolio	sarthaknegi.netlify.app
CV	View CV

University	Jaypee Institute of Information Technology
Major	Computer Science and Engineering
Degree	Bachelor of Technology
Expected Graduation	June 2026
Timezone	IST (UTC + 5:30)

11. References

[1] HEP Software Foundation. HEP Software Foundation (HSF). https://hepsoftwarefoundation.org

[2] Mashinistov, R., Gerlach, L., Laycock, P., Formica, A., Govi, G., Pinkenburg, C.: The HSF conditions database reference implementation. EPJ Web of Conferences 295, 01051 (2024). https://doi.org/10.1051/epjconf/202429501051

[3] PostgreSQL Global Development Group. PostgreSQL Documentation — EXPLAIN. https://www.postgresql.org/docs/current/sql-explain.html

[4] PostgreSQL Global Development Group. PostgreSQL Documentation — pg_stat_statements. https://www.postgresql.org/docs/current/pgstatstatements.html

[5] PostgreSQL Global Development Group. PostgreSQL Documentation — High Availability — Streaming Replication. https://www.postgresql.org/docs/current/warm-standby.html#STREAMING-REPLICATION

[6] PostgreSQL Global Development Group. PostgreSQL Documentation — CREATE INDEX (INCLUDE). https://www.postgresql.org/docs/current/sql-createindex.html

[7] BNLNPPS / HSF. Nopayloaddb Documentation (v0.3). https://bnlnpps.github.io/nopayloaddb/